

Manual do Programador

Estrutura do programa

O programa está dividido em 4 pastas:

- `bin` -> contém o ficheiro executável
 - `docs` -> contém toda a documentação do programa
 - `lib` -> contém todos os *header files* (.h)
 - `src` -> contém todos os ficheiros fonte C (.c)
-

Ficheiros e funções

- `bin`
 - Ficheiro `./proj`
- `docs`
 - Manual do Programador
 - CMakeLists.txt
- `lib`
 - `funcsMenu1.h` -> Contém a declaração das funções relativas à funcionalidade 1 do menu (introduzir dados de um novo doente)
 - Função `novoDoente()` -> utilizada para inserir um novo doente na lista e no ficheiro dos registos dos doentes, seguindo os seguintes passos: (i) atribuição de um ID que ainda não esteja em utilização; (ii) imprimir na consola todos os doentes existentes na respetiva lista; (iii) receber o input dos dados por parte do utilizador; (iv) adicionar o doente à lista dos doentes; (v) atualizar o ficheiro dos doentes (`doentes.txt`).
 - `funcsMenu2.h` -> Contém a declaração das funções relativas à funcionalidade 2 do menu (eliminar um doente existente)
 - Função `removerDoente()` -> utilizada para eliminar um doente da lista do do ficheiro dos registos dos doentes, seguindo os seguintes passos: (i) verificar se existem doentes para remover; (ii) imprimir todos os doentes registados e os respetivos IDs; (iii) receber o input do ID a eliminar; (iv) confirmar se o ID recebido é válido; (v) pedir confirmação antes de eliminar o doente; (vi) remover todos os registos do doente e removê-lo da lista dos doentes; (vii) atualizar o ficheiro dos doentes (`doentes.txt`).

- `funcsMenu3.h` -> Contém a declaração das funções relativas à funcionalidade 3 do menu (imprimir todos os doentes nos respetivos registos por ordem alfabética)
 - Função `loadDoentesOA()` -> utilizada para inserir na lista dos doentes, ordenada alfabeticamente, ponteiros para elementos da lista que contém os dados dos doentes.
 - Função `printDoentesOA()` -> utilizada para imprimir na consola todos os doentes registados na lista por ordem alfabética.
- `funcsMenu4.h` ->
- `funcsMenu5.h` ->
- `funcsMenu6.h` -> Contém a declaração das funções relativas à funcionalidade 6 do menu (registar as tensões, o peso e a altura de um determinado doente)
 - Função `novoRegisto()` -> utilizada para inserir um novo registo na lista dos registos de um determinado doente selecionado pelo utilizador.
- `funcsMisc.h` -> Contém a declaração de diversas funções utilizadas para determinadas funcionalidades ao longo do programa.
 - Função `convertToInteger()` -> converte um número dado no formato de uma string para o formato de um número inteiro (`int`), devolvendo esse mesmo valor.
 - Função `inputFunction()` -> utilizada para ler inputs do utilizador através da consola utilizando a função `fgets()` . Também inclui mecanismos para prevenir erros de leitura do input do utilizador.
 - Função `loadDoentes()` -> carrega os dados do ficheiro onde estão armazenados os dados dos doentes (`doentes.txt`) para a respetiva lista onde estarão armazenados esses dados.
 - Função `loadRegistos()` -> função que carrega os dados do ficheiro onde estão armazenados os registos dos doentes (`registos.txt`) para a respetiva lista onde estarão armazenados esses dados.
 - Função `updateDoentes()` -> utilizada para atualizar os dados dos doentes no respetivo ficheiro (`doentes.txt`), inserindo no mesmo os dados armazenados na respetiva lista.
 - Função `updateRegistos()` -> utilizada para atualizar os registos dos doentes no respetivo ficheiro (`registos.txt`), inserindo no mesmo os registos armazenados na respetiva lista.
 - Função `verifyID()` -> percorre a lista onde estão armazenados os dados dos doentes e verifica a existência de um determinado ID. Caso esse ID exista, a função irá devolver o valor 1, caso contrário, irá devolver o valor 0.
 - Função `isInteger()` -> verifica se uma string dada é um número inteiro. Em caso afirmativo, a função irá devolver o valor 1, caso contrário, irá devolver o valor 0.
 - Função `listNomes()` -> imprime na consola uma tabela com os nomes e os respetivos IDs de todos os doentes registados na lista dos doentes.

- Função `isLetter()` -> função que verifica se um dado caractere é uma letra, devolvendo o valor 1 se a condição se verificar e, em caso contrário, o valor 0. Adicionalmente, se o caractere dado for uma letra minúscula, a função irá converter o mesmo numa letra maiúscula.
- `listaDoentes.h` -> Contém a declaração das estruturas e das funções relativas à implementação de uma lista ligada para armazenar os dados dos doentes
 - Estrutura `l_noDoentes_t` -> representa um elemento da lista no qual serão armazenados os dados de um doente, o que inclui o respetivo ID (`int id`), nome (`char nome[50]`), data de nascimento (`char data_de_nascimento[50]`), número do cartão de cidadão (`char num_cc[50]`), contacto telefónico (`char contacto[50]`), email (`char email[50]`) e a outra lista ligada onde serão armazenados todos os registos relativos a esse doente (`list_registos_t registos`). Cada elemento inclui também um ponteiro para o elemento seguinte da lista.
 - Estrutura `list_doentes_t` -> armazena o número de elementos presentes na lista dos dados dos doentes e um ponteiro para o primeiro elemento da lista.
 - Função `void initListaDoentes(list_doentes_t *list)` -> inicializa a lista que irá armazenar os dados dos doentes.
 - Função `int emptyListaDoentes(list_doentes_t *list)` -> função que verifica se a lista dos dados dos doentes está vazia.
 - Função `void clearListaDoentes(list_doentes_t *list)` -> elimina todos os elementos da lista dos dados dos doentes, libertando a memória alocada para cada um dos elementos.
 - Função `void insertListaDoentes(list_doentes_t *list, int id, char nome[50], char data_de_nascimento[50], char num_cc[50], char contacto[50], char email[50])` -> utilizada para inserir um novo elemento na lista dos dados dos doentes, alocando a memória necessária para essa operação.
 - Função `void searchListaDoentes(list_doentes_t *list, int id, l_noDoentes_t **prev, l_noDoentes_t **cur)` -> utilizada para localizar uma posição na lista com base num determinado ID.
 - Função `void removeListaDoentes(list_doentes_t *list, int id)` -> utilizada para remover um doente da lista dos doentes e do ficheiro onde estão armazenados esses dados (`doentes.txt`), eliminando também todos os registos associados a esse mesmo doente.
- `listaDoentesOA.h` -> Contém a declaração das estruturas e das funções relativas à lista dos doentes ordenados alfabeticamente
 - Estrutura `l_noDoentesOA_t` -> representa um elemento da lista onde estarão armazenados um ponteiro para um elemento da lista dos dados dos doentes e outro ponteiro para o elemento seguinte da lista dos doentes ordenada alfabeticamente.

- Estrutura `list_doentesOA_t` -> armazena o número de elementos da lista dos doentes por ordem alfabética e um ponteiro para o primeiro elemento dessa mesma lista.
- Função `void initListaDoentesOA(list_doentesOA_t *list)` -> inicializa a lista dos doentes ordenada alfabeticamente.
- Função `int emptyListaDoentesOA(list_doentesOA_t *list)` -> verifica se a lista dos doentes ordenada alfabeticamente está vazia.
- Função `void clearListaDoentesOA(list_doentesOA_t *list)` -> utilizada para eliminar todos os elementos da lista dos doentes ordenada alfabeticamente, libertando a memória alocada para cada um dos elementos.
- Função `void insertListaDoentesOA(list_doentesOA_t *list, l_noDoentes_t *elem)` -> utilizada para inserir um elemento na lista dos doentes ordenada alfabeticamente.
- Função `void searchListaDoentesOA(list_doentesOA_t *list, l_noDoentes_t *elem, l_noDoentesOA_t **prev, l_noDoentesOA_t **cur)` -> utilizada para localizar uma posição na lista ordenada alfabeticamente onde pode ser inserido um elemento corretamente, por exemplo.
- Função `void removeListaDoentesOA(list_doentesOA_t *list, l_noDoentes_t *elem)` -> utilizada para remover um elemento da lista ordenada alfabeticamente.
- `listaRegistos.h` -> Contém a declaração das estruturas e das funções relativas à lista dos registos dos doentes
 - Estrutura `l_noRegistos_t` -> representa um elemento da lista onde estarão armazenados os registos de um determinado doente, incluindo o respetivo ID do doente (`int id`), a data do registo (`char data[50]`), o valor da tensão máxima (`int tensaoMax`), o valor da tensão mínima (`int tensaoMin`), o peso do doente (`int peso`), a altura do doente (`int altura`) e um ponteiro para o elemento seguinte da lista.
 - Estrutura `list_registos_t` -> armazena o número de elementos da lista dos registos de um doente e um ponteiro para o primeiro elemento da respetiva lista.
 - Função `void initListaRegistos(list_registos_t *list)` -> inicializa a função dos registos de um doente.
 - Função `int emptyListaRegistos(list_registos_t *list)` -> verifica se a lista dos registos de um doente está vazia.
 - Função `void clearListaRegistos(list_registos_t *list)` -> elimina todos os elementos da lista dos registos de um doente, libertando a memória alocada para cada um dos elementos.
 - Função `void insertListaRegistos(list_registos_t *list, int id, char data[50], int tensaoMax, int tensaoMin, int peso, int altura)` -> utilizada para inserir um novo registo de um doente na respetiva lista.

- Função `void searchListaRegistos(list_registos_t *list, int id, l_noRegistos_t **prev, l_noRegistos_t **cur)` -> utilizada para localizar um elemento na lista dos registos de um doente com base na sua tensão máxima (?)
 - Função `void removeListaRegistos(list_registos_t *list, int id)` -> remove um elemento da lista dos registos de um doente.
 - Função `void printRegistos(list_registos_t *list)` -> utilizada para imprimir na consola todos os registos de um determinado doente.
 - `src`
- As funções seguintes contém o código das respetivas funções e estruturas declaradas acima:
- `funcsMenu1.c`
 - `funcsMenu2.c`
 - `funcsMenu3.c`
 - `funcsMenu4.c`
 - `funcsMenu5.c`
 - `funcsMenu6.c`
 - `funcsMisc.c`
 - `listaDoentes.c`
 - `listaDoentesOA.c`
 - `listaRegistos.c`
 - `main.c` -> Contém o menu do programa e a seleção de todas a funcionalidades acima mencionadas.